Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of CSE

B.Tech. CSE Trimester -5

Roll Number: 2520030008
Name: Sagi Sahasra Srihasa

Generative AI
25CS2207E

LAB RECORD

2026

Week5: Token Economics

1. Aim

To develop a Python-based Token Economics Budget Planner that estimates input and output token consumption, calculates daily and monthly API costs, checks Requests Per Minute (RPM) and Tokens Per Minute (TPM) limits, and verifies whether the estimated usage stays within the configured budget using a safety margin.

2. Software Requirements

1. **Operating System:** Windows 10/11
2. **Programming Language:** Python 3.x
3. **IDE:** PyCharm / Visual Studio Code
4. **Python Library:** python-dotenv
5. **Testing Framework:** Python unittest
6. **Configuration File:** .env

3. Code

A. requirements.txt

```
None
python-dotenv
```

B. .env

```
None
INPUT_PRICE_PER_MILLION=0.30
OUTPUT_PRICE_PER_MILLION=1.20

REQUESTS_PER_DAY=2000
AVG_INPUT_TOKENS=1200
AVG_OUTPUT_TOKENS=300
PEAK_REQUESTS_PER_MINUTE=30
```

```
RPM_LIMIT=60
TPM_LIMIT=100000

MONTHLY_BUDGET_USD=50
DAILY_BUDGET_USD=2
SAFETY_MARGIN_PERCENT=20
```

C. budget.py

```
None
import os
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()


@dataclass
class FeatureBudget:
    requests_per_day: int
    avg_input_tokens: int
    avg_output_tokens: int
    peak_requests_per_minute: int
    input_price_per_million: float
    output_price_per_million: float
    rpm_limit: int
    tpm_limit: int
    monthly_budget_usd: float
    daily_budget_usd: float
    safety_margin_percent: float

    @classmethod
    def from_env(cls):
        return cls(
```

```
            requests_per_day=int(os.getenv("REQUESTS_PER_DAY",
"0")),
            avg_input_tokens=int(os.getenv("AVG_INPUT_TOKENS",
"0")),

avg_output_tokens=int(os.getenv("AVG_OUTPUT_TOKENS", "0")),

peak_requests_per_minute=int(os.getenv("PEAK_REQUESTS_PER_MINU
TE", "0")),

input_price_per_million=float(os.getenv("INPUT_PRICE_PER_MILLI
ON", "0")),

output_price_per_million=float(os.getenv("OUTPUT_PRICE_PER_MIL
LION", "0")),
            rpm_limit=int(os.getenv("RPM_LIMIT", "0")),
            tpm_limit=int(os.getenv("TPM_LIMIT", "0")),

monthly_budget_usd=float(os.getenv("MONTHLY_BUDGET_USD",
"0")),

daily_budget_usd=float(os.getenv("DAILY_BUDGET_USD", "0")),

safety_margin_percent=float(os.getenv("SAFETY_MARGIN_PERCENT",
"20"))
        )

    def calculate(self):
        if min(
            self.requests_per_day,
            self.avg_input_tokens,
            self.avg_output_tokens,
            self.peak_requests_per_minute
```

```
        ) < 0:
            raise ValueError("Usage assumptions cannot be
negative.")

        if self.safety_margin_percent < 0:
            raise ValueError("Safety margin cannot be
negative.")

        input_day = self.requests_per_day *
self.avg_input_tokens
        output_day = self.requests_per_day *
self.avg_output_tokens

        daily_cost = (
            input_day * self.input_price_per_million
            + output_day * self.output_price_per_million
        ) / 1_000_000

        monthly_cost = daily_cost * 30
        margin = 1 + self.safety_margin_percent / 100

        peak_rpm = self.peak_requests_per_minute * margin

        peak_tpm = (
            self.peak_requests_per_minute
            * (self.avg_input_tokens + self.avg_output_tokens)
            * margin
        )

        return {
            "input_tokens_day": input_day,
            "output_tokens_day": output_day,
            "total_tokens_day": input_day + output_day,
```

```
            "daily_cost_usd": daily_cost,
            "monthly_cost_usd": monthly_cost,
            "peak_rpm_with_margin": peak_rpm,
            "peak_tpm_with_margin": peak_tpm,
            "rpm_ok": peak_rpm <= self.rpm_limit,
            "tpm_ok": peak_tpm <= self.tpm_limit,
            "daily_budget_ok": daily_cost * margin <=
self.daily_budget_usd,
            "monthly_budget_ok": monthly_cost * margin <=
self.monthly_budget_usd
        }


def main():
    budget = FeatureBudget.from_env()
    result = budget.calculate()

    print("\nTOKEN ECONOMICS REPORT")
    print("-" * 35)

    print(f"Input tokens/day:
{result['input_tokens_day']:,.0f}")
    print(f"Output tokens/day:
{result['output_tokens_day']:,.0f}")
    print(f"Total tokens/day:
{result['total_tokens_day']:,.0f}")

    print(f"Estimated daily cost:
${result['daily_cost_usd']:.4f}")
    print(f"Estimated monthly cost:
${result['monthly_cost_usd']:.2f}")
```

```
    print(f"Peak RPM with margin:
{result['peak_rpm_with_margin']:.1f}")
    print(f"Peak TPM with margin:
{result['peak_tpm_with_margin']:.0f}")

    checks = {
        "RPM capacity": result["rpm_ok"],
        "TPM capacity": result["tpm_ok"],
        "Daily budget": result["daily_budget_ok"],
        "Monthly budget": result["monthly_budget_ok"]
    }

    print("\nCHECKS")

    for name, passed in checks.items():
        print(f"{name}: {'PASS' if passed else 'REVIEW'}")

    if all(checks.values()):
        print("\nRecommendation: WITHIN configured planning
limits.")
    else:
        print("\nRecommendation: REVISE assumptions, limits,
or design before build.")


if __name__ == "__main__":
    main()
```

D. test_budget.py

```
None
import unittest
from budget import FeatureBudget
```

```
class TestFeatureBudget(unittest.TestCase):

    def make_budget(self, **overrides):
        values = dict(
            requests_per_day=1000,
            avg_input_tokens=1000,
            avg_output_tokens=500,
            peak_requests_per_minute=10,
            input_price_per_million=1.0,
            output_price_per_million=2.0,
            rpm_limit=100,
            tpm_limit=50000,
            monthly_budget_usd=100,
            daily_budget_usd=5,
            safety_margin_percent=20
        )

        values.update(overrides)
        return FeatureBudget(**values)

    def test_cost_estimate(self):
        result = self.make_budget().calculate()

        self.assertAlmostEqual(result["daily_cost_usd"], 2.0)
        self.assertAlmostEqual(result["monthly_cost_usd"],
60.0)

    def test_capacity_passes(self):
        result = self.make_budget().calculate()

        self.assertTrue(result["rpm_ok"])
        self.assertTrue(result["tpm_ok"])
```

```
    def test_rpm_limit_fails(self):
        result = self.make_budget(rpm_limit=5).calculate()

        self.assertFalse(result["rpm_ok"])

    def test_monthly_budget_fails(self):
        result =
self.make_budget(monthly_budget_usd=20).calculate()

        self.assertFalse(result["monthly_budget_ok"])


if __name__ == "__main__":
    unittest.main()
```

Output:

```
TOKEN ECONOMICS REPORT
----------------------------------
Input tokens/day: 2,000,000
Output tokens/day: 600,000
Total tokens/day: 2,600,000
Estimated daily cost: $1.3200
Estimated monthly cost: $39.60
Peak RPM with margin: 36.0
Peak TPM with margin: 46800

CHECKS
RPM capacity: PASS
TPM capacity: PASS
Daily budget: PASS
Monthly budget: PASS

Recommendation: WITHIN configured planning limits.

Process finished with exit code 0
```